15 VIRTUAL FUNCTIONS AND ABSTRACTION • THEORY

03 virtual destructors override and final

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

03 — Virtual Destructors, `override`, and `final`

1. Why Base Classes Must Have a `virtual` Destructor

If you delete a derived object through a base class pointer (`base* p = new derived(); delete p;`) and `~base()` is **not** `virtual`, the C++ standard (`[expr.delete]`) states that **the behavior is Undefined Behavior** (typically only `~base()` runs and `~derived()` is never called, leaking derived resources!).

- **Rule**: Always declare `virtual ~base() {}` (or `virtual ~base() = default;`) in any polymorphic base class!
- **Can a Constructor be `virtual`?**: **NO!** A constructor cannot be `virtual` because the object's `vptr` is not yet initialized until the constructor runs.